Criteria API vs Specification

✅ **What is JPA Criteria API?**

The **JPA Criteria API** is a **type-safe**, **Java-based** way to create SQL-like queries **without writing strings**.

✅ **What is `CriteriaQuery`?**

`CriteriaQuery<T>` is a **core interface in JPA Criteria API** that represents a **query object** — like a Java version of a SQL `SELECT` statement.

Instead of writing:

```
// JPQL (string-based)
entityManager.createQuery("SELECT u FROM User u WHERE u.name = 'Houda'")
```

You write:

```
// Criteria API (type-safe)
CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<User> query = cb.createQuery(User.class);
Root<User> root = query.from(User.class);
query.select(root).where(cb.equal(root.get("name"), "Houda"));
```

Advantage	Explanation
✅ Type-safe	Uses Java types and fields, so the compiler checks errors
✅ Refactor-safe	Renaming fields in your entity won't break the query
✅ Dynamic	Easily build queries conditionally (e.g., filters)
✅ IDE-friendly	Autocomplete and no syntax errors like with string queries

🧱 **Criteria API = 4 Steps**

1. **Get a CriteriaBuilder**

   → like a "query builder" factory

2. **Create a CriteriaQuery**

   → define the result type (e.g., `User.class`)

3. **Define a Root**
   → like `FROM User u`
4. **Add conditions (WHERE, ORDER, etc.)** and run

✅ **Core Components of CriteriaQuery (Simplified)**

Component	What it is (Simple)	Example / Purpose
CriteriaBuilder	🔧 The **toolbox** to build queries	Used to create queries, conditions, etc.
CriteriaQuery	📄 The actual **query object** (like a SQL SELECT)	Defines what to SELECT, WHERE, ORDER BY...
Root	🌱 The **starting point** (the entity/table)	Like `FROM User` in SQL
Predicate	❓ The **conditions** (WHERE clauses)	Like `name = 'Ali' AND age > 20`
Expression	🧮 A **value or formula** used in a query	e.g., `SUM(price)`, `user.get("name")`

```
CriteriaBuilder cb = entityManager.getCriteriaBuilder();      // Toolbox
CriteriaQuery<User> query = cb.createQuery(User.class);       // Query object
Root<User> root = query.from(User.class);                     // FROM User

Predicate condition = cb.equal(root.get("name"), "Houda");    // WHERE name = 'Houda'
query.select(root).where(condition);                          // SELECT * FROM User WHERE ...

List<User> results = entityManager.createQuery(query).getResultList();
```

CriteriaQuery allows for specifying ordering and grouping of results.

```
cq.orderBy(cb.asc(employee.get("name")));
cq.groupBy(department.get("name"));
```

✅ **Specification – Example (Spring Data)**

Step 1: Create a `Specification<User>`

```
public class UserSpecifications {
    public static Specification<User> hasName(String name) {
        return (root, query, cb) -> cb.equal(root.get("name"), name);
    }

```

```
    public static Specification<User> hasAge(Integer age) {
        return (root, query, cb) -> cb.equal(root.get("age"), age);
    }
}
```

Step 2: Use it in a repository

```
List<User> users = userRepository.findAll(
    Specification.where(hasName("Houda")).and(hasAge(25))
);
```

You should use...	When...
Criteria API	You're writing manual queries, not using Spring Data
Spring Data Specification	You're using Spring Data and want clean, dynamic filter queries

> "Spring Data `Specification` is a wrapper around the JPA Criteria API.
> I use Criteria API when I'm not using Spring Data, and I use Specifications when I want to build dynamic, composable queries in Spring Data JPA repositories."

Quelle est la **signature** de la méthode à implémenter dans une `Specification<T>` ?

🎯 **Réponse claire :**

> L'interface `Specification<T>` contient **une seule méthode** à implémenter :

```
Predicate toPredicate(Root<T> root, CriteriaQuery<?> query, CriteriaBuilder criteriaBuilder);
```

🧠 Explication des paramètres :

Paramètre	Rôle
`Root<T> root`	Représente **l'entité de base** (ex. `Product`)
`CriteriaQuery<?> query`	L'objet qui représente la requête complète
`CriteriaBuilder criteriaBuilder`	Utilisé pour **construire les conditions** (`equal`, `like`, etc.)

✅ Exemple simple :

```java
public static Specification<Product> hasCategory(String category) {
    return (root, query, cb) -> cb.equal(root.get("category"), category);
}
```

💻 CHALLENGE — Recherche multi-critères sur entités liées

```java
@Entity
public class Author {
    @Id
    @GeneratedValue
    private Long id;
    private String name;
}

@Entity
public class Book {
    @Id
    @GeneratedValue
    private Long id;
    private String title;
    private int year;

    @ManyToOne
    private Author author;
}
```

🔸 **Objectif :**

Écris une `Specification<Book>` qui :

- filtre les livres dont **le titre contient un mot-clé**
- **ET** l'auteur a un nom qui commence par une certaine valeur
- **ET** l'année est **supérieure ou égale à** une année donnée (ex: 2010)

✅ Version corrigée :

```java
public static Specification<Book> searchBooks(String titleKeyword, String authorPrefix, int minYear) {
    return (root, query, cb) -> cb.and(
        cb.like(root.get("title"), "%" + titleKeyword + "%"),
        cb.like(root.join("author").get("name"), authorPrefix + "%"),
        cb.greaterThanOrEqualTo(root.get("year"), minYear)
    );
}
```

🔥 CHALLENGE BONUS — API de recherche avec filtres, pagination, et tri

```java
@Entity
public class Book {
    @Id @GeneratedValue
    private Long id;

    private String title;
    private int year;
    private String genre;
```

```
}
```

🎯 **Objectif :**

Tu dois écrire une **méthode de service** qui retourne une liste paginée et triée de livres, selon ces paramètres :

Paramètre	Description
`String keyword`	(facultatif) : filtrer par titre contenant
`String genre`	(facultatif) : filtrer par genre exact
`int minYear`	(facultatif) : ne retourner que les livres publiés **après ou égal à** cette année
`int page`	numéro de page
`int size`	taille de page
`String sortBy`	nom du champ de tri (`title` , `year` , etc.)
`boolean asc`	tri ascendant ou descendant

✅ Version corrigée

> ℹ `JpaSpecificationExecutor<T>` ?
>
> C'est une interface fournie par Spring Data que tu ajoutes à ton repository pour activer les **recherches dynamiques avec Specification**.

```java
public interface BookRepository extends JpaRepository<Book, Long>, JpaSpecificationExecutor<Book>
{
}
```

```java
public class BookSpecification {

    public static Specification<Book> hasKeyword(String keyword) {
        return (keyword == null || keyword.isEmpty()) ? null :
            (root, query, cb) -> cb.like(cb.lower(root.get("title")), "%" +
keyword.toLowerCase() + "%");
    }

    public static Specification<Book> hasGenre(String genre) {
        return (genre == null || genre.isEmpty()) ? null :
            (root, query, cb) -> cb.equal(root.get("genre"), genre);
    }

    public static Specification<Book> hasMinYear(Integer minYear) {
        return (minYear == null) ? null :
            (root, query, cb) -> cb.greaterThanOrEqualTo(root.get("year"), minYear);
```

```
    }
}
```

```java
public Page<Book> searchBooks(String keyword, String genre, Integer minYear,
                              int page, int size, String sortBy, boolean asc) {

    Sort sort = asc ? Sort.by(sortBy).ascending() : Sort.by(sortBy).descending();
    Pageable pageable = PageRequest.of(page, size, sort);

    Specification<Book> spec = Specification.where(BookSpecification.hasKeyword(keyword))
        .and(BookSpecification.hasGenre(genre))
        .and(BookSpecification.hasMinYear(minYear));

    return bookRepository.findAll(spec, pageable);
}
```